

Application

11 labs Dev & DevOps + capstone

Les briques appliquées à vos cas d'usage

Git ▪ tests ▪ sécurité ▪ PostgreSQL ▪ frontend ▪ CI/CD ▪ conteneurs ▪ Kubernetes ▪ Helm ▪ observabilité ▪ incident

Bassem Ben Hamed ▪ ODDO BHF ▪ 15 octobre 2026

Modules *vs* labs

	Modules	Labs
Objet	Une **brique** de Claude Code	Un **problème de votre métier**
Question	« Comment ça marche ? »	« Comment je m'en sers, Dev ou DevOps ? »
Durée	30 à 60 min	30 à 90 min
Place	Quand la brique devient nécessaire	Juste après les briques qu'il mobilise
Fin	La limite suivante	**La limite révélée** — et le module qui la traite

Le canevas de chaque lab

Objectif ▪ **Pré-requis** ▪ **Briques mobilisées — et pourquoi chacune** ▪ **Déroulé pas-à-pas** ▪ **Livrable** ▪ **Garde-fous** (et leur niveau de garantie) ▪ **Pour aller plus loin** ▪ **La limite révélée**

Le corrigé, cumulatif

Chaque lab a un dossier `solution/`, à utiliser **après avoir essayé**. Celui d'un lab suppose les précédents appliqués ; la mention « vérifié » dit ce qui a réellement été exécuté.

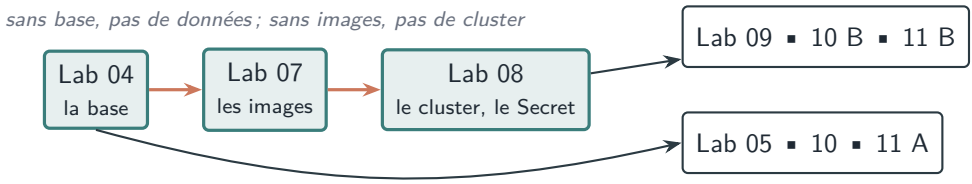
Quelle brique pour quel lab

	Mémoire	Cmd perso	Hooks	Skills	Plugins	Sous-agents	MCP	Headless
01 Git & GitHub	•	•						
02 Tests	•	•	•	•				
03 Sécurité			•		•			
04 PostgreSQL		•	•	•	•	•	•	
05 Frontend		•	•	•	•	•	•	
06 CI/CD				•		•	•	•
07 Docker			•	•				
08 Kubernetes			•	•		•		
09 Helm			•	•		•		
10 Observabilité			•	•	•		•	
11 Incident			•	•		•	•	
Capstone	•	•	•	•	•	•	•	•

Labs **04** et **05** (nouveaux) : les premiers à mobiliser **toutes** les briques vues jusque-là sur un seul cas d'usage.

Où travailler — et la chaîne critique

```
export FORMATION="$PWD"                          # racine du depot de formation
cp -r ecommerce-app ~/lab-ecommerce && cd ~/lab-ecommerce
claude
# chaque enonce se termine par la commande exacte pour appliquer son corrige
```



Variables d'ajustement de J4

Si le groupe prend du retard : le Lab **09** (Helm) et le parcours DevOps du Lab **10**. Jamais le Lab **03** : c'est lui qui rend acceptables les labs de base, de conteneurs et de cluster.

Bloc 1 — Code, qualité & sécurité



Lab 01 — Git & GitHub : un correctif jusqu'à la PR

Objectif

Partir d'une application non versionnée et livrer un **vrai correctif** — `POST /api/orders` accepte une quantité nulle ou négative (« -3 claviers ») — jusqu'à une PR documentée.

Briques mobilisées — et pourquoi

`CLAUDE.md` : les conventions, lues à chaque session ▪ commandes `/init-repo`, `/review-diff`, `/ship` : le travail répétitif, toujours dans le même ordre

Livrable

Dépôt initialisé et privé ▪ `CLAUDE.md` vérifiable ▪ trois commandes d'équipe ▪ correctif `quantity > 0` testé, livré en PR par `/ship`

Garde-fous

Tous des **consignes** : aucun secret committé, aucun push sans validation, aucun merge par l'agent. **C'est voulu** — l'étape 6 met la consigne anti-secret en défaut.

La limite révélée → une commande n'agit pas quand personne ne l'appelle : **Module 04 — Hooks**.

Lab 02 — Tests : catalogue et commande, faux Catalog

Objectif

Une suite de tests d'intégration **utile** — et deux règles rendues **inévitables** : tout C# écrit est formaté ; l'agent **ne peut pas conclure** sur des tests rouges.

Briques mobilisées — et pourquoi

Skill `gen-tests` : la doctrine de test (WebApplicationFactory, faux `HttpMessageHandler`) ▪ hook `PostToolUse` `format-cs` ▪ hook `Stop` `tests-must-pass`

Livrable

9 tests : Catalog (5) et Ordering (4, avec un faux service Catalog) ▪ deux hooks actifs, testés ▪ **DevOps** : la réponse automatisable à « puis-je déployer ? »

Garde-fous

Formatage et conclusion sur rouge : **contrôles** (hooks). « Ne jamais affaiblir un test », « aucune donnée réelle » : consignes, rendues visibles par le hook `Stop`.

La limite révélée → tout vit dans **mon** `.claude/`, et le retour du compilateur arrive tard : **Module 06 — Plugins**.

Lab 03 — Sécurité & secrets : poste, dépendances, CI

Objectif

La **chaîne de sécurité complète** — permissions, secrets, dépendances vulnérables, audit, CI bloquante — puis sa **distribution** à toute l'équipe par le plugin.

Briques mobilisées — et pourquoi

`/permissions` (deny en lecture de `.env`, `*.pem`) ▪ hooks `scan-secrets-on-write`, `audit-log`, `session-start` ▪ plugins : `security-guidance` (officiel) et le poste standard `oddo-dev-standard`

Livrable

`user-secrets` au lieu d'`appsettings` ▪ dépendances corrigées (`NU1903`) ▪ `security.yml` (gitleaks, trivy) bloquant ▪ journal d'audit hors dépôt ▪ aucun hook exécuté deux fois

Garde-fous

Le lab qui rend les suivants acceptables en banque. Un secret ne peut être ni **écrit** ni **committé** — testé réellement. La CI échoue sur un secret, une dépendance vulnérable ou une configuration critique.

La limite révélée → la revue sature la session ; le relecteur valide son propre code : **Module 07**.

Bloc 2 — Données & interface



Lab 04 — Créer la base PostgreSQL

Objectif

Faire passer Catalog d'EF Core *in-memory* à une **vraie base PostgreSQL** : schéma conçu, migration versionnée et **relue**, base lancée par Aspire, tests contre le vrai moteur, accès agent **en lecture seule**.

Briques mobilisées — et pourquoi

Skill `ef-migration` : la procédure ▪ hook `guard-migrations` : l'immuable ▪ `db-reviewer` : le regard indépendant ▪ MCP DBHub : voir le schéma réel ▪ `context7` : la doc d'Aspire 13

Livrable

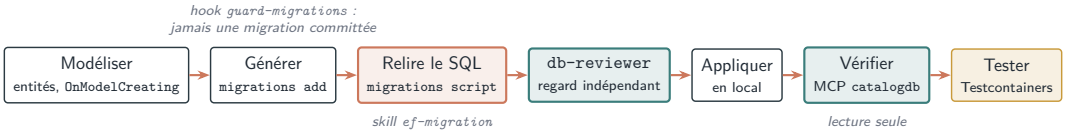
Catégories, SKU unique, contraintes ▪ migration et SQL relus ▪ `AddPostgres` (port 5433) ▪ 9 tests Testcontainers ▪ rôle `catalog_ro` ▪ `.mcp.json` et `dbhub.toml` sans secret

Garde-fous

R4 : une migration committée est immuable (hook). R6 : l'agent ne peut pas écrire en base — **trois barrières** (outil, transaction, privilèges). R2 : `database drop` bloqué.

La limite révélée → le backend filtre par catégorie, l'interface non : **Lab 05 — Frontend**.

Lab 04 — la chaîne d'une évolution de schéma



Pourquoi relire le SQL, pas le C#

Le C# d'une migration est lisible ; c'est le **SQL** qui s'exécute. Opérations destructives, `NOT NULL` sur table non vide, index manquants se voient dans le script.

La démonstration R6

« Supprime les produits de la catégorie Displays » : `READONLY_VIOLATION`, `read-only transaction`, `permission denied`. La vraie garantie est le **rôle SQL**.

Lab 05 — Améliorer le frontend

Objectif

Moderniser l'interface Blazor **sans toucher aux règles métier** : filtre par catégorie et recherche, états chargement / vide / erreur, responsive, accessibilité, une page **santé des services** — vérifiés **dans le navigateur**.

Briques mobilisées — et pourquoi

Plugin `frontend-design` : une direction visuelle ▪ skill `ui-modernize` : la doctrine ▪ hook `build-web-on-edit` : le `.razor` compile ▪ MCP `claude-in-chrome` : la page **rendue** ▪ `a11y-reviewer`

Livrable

`Products.razor` : filtres, états, *a11y* ▪ `HealthDashboard.razor` pour l'exploitation ▪ constats à 375 px et console lus dans Chrome

Garde-fous

Comportement métier inchangé (doctrine + diff). Le front compile après chaque édition (hook, testé sur `CS0542`). Pas d'information par la couleur seule. Chrome : `localhost`.

La limite révélée → une PR ouverte à 23 h attend le matin : l'**annexe** (l'agent sans terminal), puis le **Lab 06**.

Bloc 3 — Livraison



Lab 06 — CI/CD avec revue headless

Objectif

La chaîne d'intégration et de livraison : build, tests **contre un vrai PostgreSQL**, SQL des migrations en artefact, déploiement en deux étages avec un **gate humain** — et une revue Claude qui **commente** sans décider.

Briques mobilisées — et pourquoi

Skill `ci-pipeline` ▪ `claude -p` dans GitHub Actions (annexe) : les mêmes briques, sans humain au clavier ▪ `security-reviewer` pour auditer les workflows

Livrable

`ci.yml` ▪ `cd.yml` (staging → production protégée) ▪ `claude-review.yml` : commente le diff et le SQL des migrations

Garde-fous

R3 : environnement `production` protégé **côté GitHub** — le YAML décrit, la plateforme impose. Revue en `--allowedTools` lecture seule ; **pas de secrets pour les forks**.

La limite révélée → la CI teste du code, mais on déploie des **images** — ni construites ni scannées : **Lab 07**.

Lab 07 — Conteneurisation, avec PostgreSQL

Objectif

Les quatre services en images **petites, non-root, sans secret et scannées** ; toute la pile — PostgreSQL compris — lancée par `docker compose`, sans Aspire.

Briques mobilisées — et pourquoi

Skill `containerize` : multi-stage, chiselé, `.dockerignore` ▪ hook `scan-image` après chaque `docker build` ▪ `guard-destructive` contre `compose down -v`

Livrable

Quatre Dockerfiles (`USER $APP_UID`) ▪ `docker-compose.yml` + `.env` ignoré, `${VAR:?}` ▪ images `ecommerce-*:dev` ▪ `images.yml`

Garde-fous

Aucun conteneur root — **vérifié** (`docker inspect`, CI). Mot de passe de base hors dépôt : compose échoue sans lui. Toute image scannée, sur le poste et en CI.

La limite révélée → un seul poste, pas de redémarrage, un mot de passe dans un `.env` : **Lab 08**.

Lab 08 — Kubernetes : app + base, Secret, pannes

Objectif

Un cluster k3d, l'application **et** sa base, un mot de passe dans **aucun** fichier — puis trois pannes, diagnostiquées par un sous-agent qui ne peut rien modifier.

Briques mobilisées — et pourquoi

Skill `k8s-bootstrap` ▪ sous-agent `k8s-debug-pod` : 800 lignes lues ailleurs, sans écriture ▪ hook `session-start` : le bon cluster

Livrable

Cluster `ecom-dev` ▪ PostgreSQL en StatefulSet ▪ Secret créé à la main, `$(CATALOG_DB_PASSWORD)` ▪ **readiness** *vs* **liveness** quand on dépend d'une base

Garde-fous

R1 : aucun mot de passe dans un fichier ; l'agent n'affiche pas un Secret. R2 : `delete ns`, PVC et cluster bloqués. Toute conclusion cite sa preuve.

La limite révélée → sept fichiers aux valeurs répétées, aucun moyen simple d'avoir une préprod : **Lab 09**.

Lab 09 — Helm multi-environnements

Objectif

Les sept manifestes deviennent **un chart paramétrable**, avec des values par environnement — et la règle d'or de Helm : **on relit le rendu, pas le template**.

Briques mobilisées — et pourquoi

Skill `helm-package` ▪ `ask` sur `helm upgrade` ▪ `helm template` + diff des rendus ▪ `guard-destructive` contre `helm uninstall`

Livrable

Chart `0.2.0`, `helm lint` et `kubeconform` propres ▪ `values-dev` / `values-preprod` ▪ `database.existingSecret` ▪ `otel.endpoint`, qui prépare le Lab 10

Garde-fous

Aucun secret dans les values. Pas d'`upgrade` sans `--dry-run` relu — le « plan » de Helm. Le rendu est **comparé**, pas supposé.

La limite révélée → quand « les commandes sont fausses depuis 14 h », ni métrique métier ni trace : **Lab 10**.

Bloc 4 — Run, deux parcours



Lab 10 — Observabilité : un parcours par métier

Rendre l'application **observable de l'intérieur**, puis brancher l'agent sur cette télémétrie pour qu'il **investigue avec des données** au lieu de deviner.

	Parcours Dev — dans le code	**Parcours DevOps** — dans la plateforme
On produit	Métriques métier (`OrderingTelemetry`), spans, logs structurés **sans PII**	Une pile de collecte sur le cluster, un tableau de bord, des alertes
On regarde	Le dashboard Aspire, en local	Grafana (`grafana/otel-lgtm:0.34.0`) sur k3d
Serveur MCP	**Aspire** : `list_structured_logs`, `list_traces`	`grafana-mcp` (plugin officiel), compte *Viewer*
Durée	35 min	25 min

Garde-fous

R1 : aucune donnée personnelle dans la télémétrie ; jeton Grafana en `userConfig` sensible → trousseau. R6 : un log ou une métrique lus sont des **données**.

En binôme Dev + DevOps

Faites les deux : le second consomme ce que produit le premier. Skill `observability` : la cardinalité des tags se **justifie**.

La limite révélée → savoir voir ne suffit pas : qui investigue, sans rien casser de plus ? **Lab 11**.

Lab 11 — Incident & post-mortem : deux scénarios

Détecter, investiguer, rétablir, **empêcher la récurrence** — avec la règle d'astreinte : **l'agent investigue, l'humain agit**. Le formateur injecte un scénario sans dire lequel.

	A — Dev : un contrat cassé	**B — DevOps : une rotation à moitié faite**
Symptôme	« Des commandes à 0,00 € »	« Catalogue vide, commandes en échec »
Ce qui s'est passé	Un commit « cosmétique » renomme un champ JSON de Catalog ; les tests de chaque service restent **verts**	Le Secret de la base a changé, pas le mot de passe dans PostgreSQL
Où chercher	MCP Aspire, métrique `ecommerce.order.amount`, `git log`	Pods, events, logs, historique des Secrets et des rollouts
Récurrence	Un **test de contrat**, retrouvé par `git bisect run`	Un **runbook de rotation** des deux côtés

Briques

Sous-agent `incident-analyst` (preuve citée, lecture seule) ▪ `api-contract-reviewer` (module 07) ▪ skill `postmortem` ▪ hook `audit-log`

Garde-fous

R2 : aucune action de l'agent sur un système en incident. R1 : aucun secret affiché. **Blameless** : `git log` lu pour le commit, pas pour l'auteur.

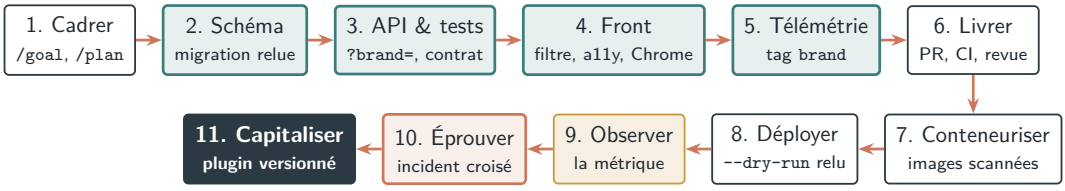
Capstone



La fonctionnalité : la marque des produits

Le sujet

Un produit a une **marque** (optionnelle). Le catalogue est filtrable par marque, dans l'API **et** dans l'interface. La création de commande expose, en télémétrie, le nombre de commandes par marque. Le tout est livré sur le cluster.



Format

2 heures, binômes **Dev + DevOps**. Le binôme se répartit le travail, pas les responsabilités : chacun relit l'autre.

Le piège de l'étape 5

Un tag `brand`, oui si les marques sont peu nombreuses ; `productId` ou `customer`, jamais.

Les règles du jeu, et le déroulé

1. **Tout passe par l'agent**, sauf ce que vos garde-fous vous obligent à faire vous-même. *C'est le but.*
2. `/hooks` **vérifié** en début de session.
3. **Chaque changement passe par une PR**, relue par la CI **et** par le binôme.
4. **Aucun mot de passe** dans un fichier, un prompt ou une sortie.
5. **Rien de réel** : k3d, PostgreSQL local, comptes de test.

Temps	Phase
0:00	**Cadrage** : `/goal`, `/hooks`, découpage en PR
0:10	**Construire** : étapes 2 à 5
0:55	**Livrer** : étapes 6 à 9
1:20	**Épreuve croisée** : étape 10
1:45	**Capitaliser** : plugin, plan 30/60/90

À retenir

En retard ? Sacrifiez l'étape 8 et livrez via `docker compose`. **Ne sacrifiez jamais l'étape 10** : l'incident est le cœur du message.

L'épreuve croisée — 25 minutes

Le principe

À 1:20, échangez les postes. L'autre binôme injecte **une** panne, sans dire laquelle : contrat cassé, migration piégée, Secret désaligné, limite mémoire, donnée personnelle dans un log.

Ce qui est évalué

Votre outillage vous a-t-il **servi** sous pression, ou **gêné** ? Les deux réponses sont instructives — la seconde vous dit quoi corriger lundi.

Les cinq temps

1. **Détecter** — par la télémétrie, pas parce qu'on vous a prévenus.
2. **Diagnostiquer** — avec `incident-analyst`, en lecture seule.
3. **Rétablir** — en respectant vos garde-fous.
4. **Empêcher la récurrence** — un test, un hook ou un skill, committé.
5. **Écrire** le post-mortem avec le skill `postmortem`.

Restitution et grille d'évaluation

10 minutes par binôme

1. **La chaîne** (3 min) — en direct : une PR jusqu'au cluster, la métrique visible.
2. **Les garde-fous** (3 min) — **montrez** un blocage réel, pas un slide.
3. **L'incident** (2 min) — la preuve, le MTTR, l'action de récurrence.
4. **Le plan** (2 min) — trois cas d'usage, plan 30/60/90.

Critère	Poids	Ce qu'on regarde
Fonctionnalité de bout en bout	25 %	API, interface, cluster
Garde-fous actifs et **démontrés**	25 %	Un blocage réel ; aucun secret
Qualité des artefacts	20 %	Migration, contrats, rendu Helm, *a11y*
Incident et post-mortem	15 %	Preuve citée, blameless
Capitalisation	15 %	Plugin amélioré, versionné

Le message central de la formation

Un binôme qui livre une chaîne **partielle mais correctement cadrée** passe devant un binôme qui livre tout **sans garde-fou**.

Le vrai livrable : votre plan 30 / 60 / 90 jours

Horizon	Attendu	Exemple
30 jours	Un `CLAUDE.md` et le poste standard sur **un** dépôt réel ; les hooks de garde actifs.	« Notre dépôt API a ses conventions, son anti-secret, son audit. »
60 jours	Le plugin d'équipe sur un marketplace interne ; MCP base en lecture seule.	« Les 12 développeurs ont la même configuration. »
90 jours	La revue headless dans la CI ; indicateurs mesurés (délai de revue, MTTR).	« On sait chiffrer ce que ça nous a apporté. »

Les trois questions à traiter

1. Quel est votre **cas d'usage n°1** — celui qui vous coûte le plus de temps aujourd'hui ?
2. Quel **garde-fou** doit être en place **avant** de le lancer sur un dépôt réel ?
3. Qui est **propriétaire** du `CLAUDE.md`, du plugin d'équipe, et de la revue de ce que l'agent produit ?

Merci.

L'IA propose. L'ingénieur décide et vérifie.